CodePlay : A Fun & Interactive Game Builder for Kids

A Project Report

Submitted by

G Harshith Sai – AP23110010170

M Dhanesh Sai– AP23110010275

K Jathin – AP23110010007

T Koushik – AP23110010234

Under the Supervision of

Mr. B. L. V. SIVA RAMA KRISHNA
Assistant Professor
Department of Computer Science and Engineering
SRM University-AP

In partial fulfilment for the requirements of the Web Technology(CSE210) Project

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY-AP

NEERUKONDA

MANAGALAGIRI - 522503

ANDHRA PRADESH, INDIA

APRIL-2025

CERTIFICATE

This is to certify that the project work entitled **"CODEPLAY"** is a Bonafide record of project work carried out by the following students:

- **Mr. G.Harshith Sai** (Roll No.: AP23110010170)
- **Mr. M.Dhanesh Sai** (Roll No.: AP23110010275)
- **Mr. K.Jathin** (Roll No.: AP23110010007)
- **Mr. T.Koushik** (Roll No.: AP23110010234)

from the **Department of Computer Science and Engineering, SRM University-AP.** The students conducted this project work under my supervision during the period **January 2025 to April 2025**.It is further certified that, to the best of my knowledge, this project has not previously formed the basis for the award of any degree or any similar title to this or any other candidate.

This is also to certify that the project work represents the **teamwork** of the candidates.

Station: Mangalagiri
Date: 23 Apr 2025

Mr. B. L. V. Siva Rama Krishna
Assistant Professor
Department of Computer Science & Engineering
SRM University-AP
Andhra Pradesh.

Table of Contents

1. Introduction

The CodePlay web application is a dynamic and innovative platform designed to democratize game development by enabling users, particularly beginners, to create, edit, and play browser-based games. The system offers a structured environment where users can select from predefined game templates—Maze Runner, Jump Adventure, and Catch the Fruit—and customize them using JavaScript and HTML. By integrating user authentication, a MySQL database for persistent storage, and a responsive interface built with Bootstrap, CodePlay ensures accessibility and ease of use for a wide audience, including students, hobbyists, and aspiring developers.

The platform's primary goal is to lower the barriers to entry in game development, a field often perceived as complex and resource-intensive. CodePlay achieves this through an intuitive interface that combines a code editor, live preview canvas, and game management dashboard. Users can register, log in, select a template, write code, save their games, and revisit them later. The application is built using PHP for server-side logic, MySQL for data management, and Bootstrap 5.3 for front-end styling, ensuring a modern and cohesive user experience.

CodePlay's educational value lies in its ability to teach fundamental programming concepts through hands-on game creation. The templates provide starter code, allowing users to experiment with game mechanics like player movement, collision detection, and scoring. The platform also fosters creativity by enabling users to modify templates or create custom games. Its responsive design ensures accessibility across devices, from desktops to mobile phones, making it a versatile tool for learning and experimentation.

The project's significance extends beyond its technical implementation. By providing a free, web-based solution, CodePlay addresses the need for accessible educational tools in computer science. It aligns with the growing demand for interactive learning platforms that combine coding practice with immediate feedback, helping users build confidence and skills in a supportive environment.

The development of CodePlay reflects a thoughtful approach to user experience, security, and functionality. The use of PHP sessions for authentication, PDO for secure database interactions, and Bootstrap for styling demonstrates a commitment to modern web development practices. However, the absence of certain files (e.g., cg.html, editor.html) and potential security considerations (e.g., JavaScript execution risks) suggest areas for improvement, which will be explored later in this report.

This report provides a comprehensive analysis of the CodePlay project, covering its purpose, technical implementation, and potential enhancements. It is structured to address the problem definition, objectives, requirements, design, implementation, database, and expected outputs, offering a holistic view of the system's capabilities and limitations.

2. Problem Definition

Game development is a multidisciplinary field that combines programming, design, and creativity. However, it poses significant challenges for beginners and non-experts due to several factors:

1. **Complexity of Tools**: Professional game development environments like Unity or Unreal Engine require significant setup, hardware resources, and expertise, making them intimidating for novices. Even simpler tools like Scratch, while user-friendly, may lack the flexibility needed for more advanced projects.
2. **Steep Learning Curve**: Understanding game mechanics—such as physics, collision detection, and user input handling—requires knowledge of programming concepts that may be unfamiliar to beginners. Without guided examples, learners may struggle to translate ideas into functional code.
3. **Lack of Integrated Environments**: Many development platforms do not offer an all-in-one solution for coding, testing, and saving projects. Users often need separate tools for writing code, debugging, and deploying games, which can be cumbersome and discouraging.
4. **Data Persistence**: For a platform to support user-created content, it must securely store and retrieve projects. Implementing a reliable database system while ensuring user privacy and data integrity is a non-trivial challenge.
5. **Accessibility**: Many game development tools are desktop-based or require paid licenses, limiting access for users with budget constraints or those using mobile devices.

CodePlay addresses these challenges by providing a web-based platform that simplifies the game development process. It offers prebuilt templates to guide users, an integrated editor with real-time preview, and a database for storing games. The platform's browser-based nature eliminates the need for specialized software, making it accessible to anyone with an internet connection. By focusing on JavaScript and HTML—widely taught languages—CodePlay ensures that users can leverage familiar skills while learning game-specific concepts.

The problem is further compounded by the lack of educational tools that balance simplicity with flexibility. CodePlay fills this gap by providing a structured yet customizable environment, enabling users to start with templates and progress to more complex projects. Its emphasis on user authentication and data security ensures a safe and personalized experience, addressing common concerns in web-based applications.

3. Problem Statement

The core challenge is to design and implement a web-based game development platform that empowers users with minimal programming experience to create, edit, and play browser-based games. The system must address the following requirements:

- **Intuitive Interface**: Provide a user-friendly interface for selecting game templates, writing code, and previewing results, minimizing the learning curve for beginners.
- **Secure Authentication**: Implement robust user registration and login mechanisms to protect user accounts and ensure only authorized access to game data.
- **Real-Time Feedback**: Enable users to see the results of their code immediately through a live preview canvas, facilitating experimentation and debugging.
- **Game Management**: Allow users to save, edit, delete, and view their games in a personalized dashboard, ensuring a seamless workflow.
- **Responsive Design**: Ensure the platform is accessible and visually consistent across devices, including desktops, tablets, and smartphones.
- **Data Storage**: Integrate a secure database to store user profiles and game data, with efficient retrieval and update mechanisms.
- **Educational Support**: Offer templates that teach game development concepts, such as player movement, scoring, and collision detection, in an engaging manner.

The solution must balance simplicity for novice users with sufficient flexibility for customization, enabling progression from template-based games to original projects. It should also prioritize security to prevent vulnerabilities like SQL injection or cross-site scripting (XSS), given the platform's reliance on user-generated code. Performance is critical, as users expect fast page loads and smooth code execution. Finally, the system must be maintainable and scalable to support a growing user base and potential feature expansions, such as additional templates or multiplayer capabilities.

4. Objectives

The CodePlay project is driven by the following objectives, each designed to address the identified challenges and deliver a robust, user-centric platform:

1. **Simplify Game Creation**: Develop an intuitive platform where users can select from three predefined game templates (Maze Runner, Jump Adventure, Catch the Fruit) and customize them using JavaScript and HTML, reducing the complexity of game development.
2. **Ensure Secure Authentication**: Implement user registration and login with password hashing and session management to protect user accounts and maintain data privacy.
3. **Enable Game Management**: Provide a dashboard where users can save, edit, delete, and view their games, ensuring a seamless and organized workflow.
4. **Support Real-Time Preview**: Integrate a code editor with a live preview canvas, allowing users to test their games instantly and iterate on their code.
5. **Deliver Responsive Design**: Create a visually appealing and responsive interface using Bootstrap, ensuring compatibility across desktops, tablets, and mobile devices.
6. **Implement Robust Data Storage**: Use a MySQL database with PDO for secure storage and retrieval of user profiles and game data, supporting scalability and reliability.
7. **Promote Learning**: Offer educational templates that introduce key game development concepts, such as player movement, collision detection, and scoring, in an accessible and engaging format.
8. **Enhance User Experience**: Incorporate modern design elements, such as background videos and hover effects, to create an engaging and professional interface.
9. **Ensure Security and Performance**: Use best practices like prepared statements, input validation, and optimized code to prevent vulnerabilities and ensure fast load times.

These objectives collectively aim to create a platform that is both functional and educational, catering to users of varying skill levels while maintaining high standards of security, usability, and performance.

5. Software Requirements Specifications

Functional Requirements

The CodePlay platform is designed to meet the following functional requirements, ensuring a comprehensive and user-friendly experience:

1. **User Authentication**:
    - **Registration**: Users can sign up with a full name, email, and password (minimum 8 characters, including uppercase, lowercase, and numbers).
    - **Login**: Users can log in with their email and password, with session-based access control to protect private pages.
    - **Logout**: Users can log out, clearing their session data and redirecting to the login page.
    - **Security**: Passwords are hashed using PHP's password_hash() function, and PDO prepared statements prevent SQL injection.
2. **Game Template Selection**:
    - Users can choose from three templates: Maze Runner, Jump Adventure, and Catch the Fruit, each with predefined JavaScript code.
    - Templates are displayed as cards with images and descriptions, linking to the editor with the selected template's starter code.
3. **Game Editor**:
    - Provides a textarea for writing JavaScript/HTML code, with fields for game title and template type.
    - Includes a canvas element for real-time preview of the game, updated when the user clicks "Run Code."
    - Supports saving new games or updating existing ones, with validation to ensure all fields are filled.
    - Allows editing of previously saved games, pre-populating the editor with existing data.
4. **Game Management**:
    - Displays a list of user's saved games with details like title, template type, creation date, and last update.
    - Offers options to edit, delete, or play games, with confirmation prompts for deletion.
    - Supports sorting games by last updated timestamp for easy access.
5. **Game Execution**:
    - Users can play saved games in a dedicated interface with a canvas element.
    - Supports standard controls: arrow keys or WASD for movement, spacebar for jumping or interaction.
    - Displays error messages if the game code fails to execute.

Non-Functional Requirements

The platform must adhere to the following non-functional requirements to ensure quality and reliability:

1. **Performance**:
    - Pages should load within 2 seconds under normal network conditions.
    - Database queries should execute within 100 milliseconds for typical operations.
    - Game code execution in the canvas should maintain a frame rate of at least 30 FPS for smooth gameplay.
2. **Security**:
    - Passwords are securely hashed using password_hash() and verified with password_verify().
    - PDO prepared statements prevent SQL injection attacks.
    - Session management ensures users cannot access protected pages without authentication.
    - User inputs (e.g., game title, code) should be sanitized to prevent XSS attacks.
3. **Usability**:
    - The interface is intuitive, with clear navigation and consistent styling via Bootstrap 5.3.
    - Forms include validation feedback (e.g., password requirements, required fields).
    - Tooltips or help text guide users through the editor and game controls.
4. **Scalability**:
    - The database supports multiple users and games, with efficient indexing for query performance.
    - The application can handle at least 100 concurrent users without significant performance degradation.
5. **Compatibility**:
    - Supports modern browsers (Chrome, Firefox, Safari, Edge) on desktop and mobile devices.
    - Responsive design ensures usability on screens from 320px to 1920px wide.

Hardware Requirements

- **Server**:
    - Web server with 2GB RAM, 2 CPU cores, and 20GB storage.
    - PHP 7.4+ and MySQL 5.7+ installed.
    - Stable internet connection for hosting.
- **Client**:
    - Device with a modern web browser (e.g., Chrome 90+, Firefox 88+).
    - Minimum 2GB RAM and internet access.

Software Requirements

- **Server-Side**:
  - PHP 7.4 or higher for server-side logic.
  - MySQL 5.7 or higher for database management.
  - Apache or Nginx web server for hosting.
- **Client-Side**:
  - HTML5 for structure, CSS3 for styling, JavaScript for interactivity.
  - Bootstrap 5.3 for responsive design and components.
- **Development Tools**:
  - Text editor (e.g., Visual Studio Code, Sublime Text).
  - Local development environment (e.g., XAMPP, WAMP, MAMP).
  - Browser developer tools for debugging.

Constraints

- The platform relies on JavaScript execution in the browser, which may pose security risks if user code is not properly sandboxed.
- The database assumes a single server setup; scaling to multiple servers requires additional configuration.
- The background videos (e.g., nature.mp4) may increase page load times on slow connections, necessitating optimization.

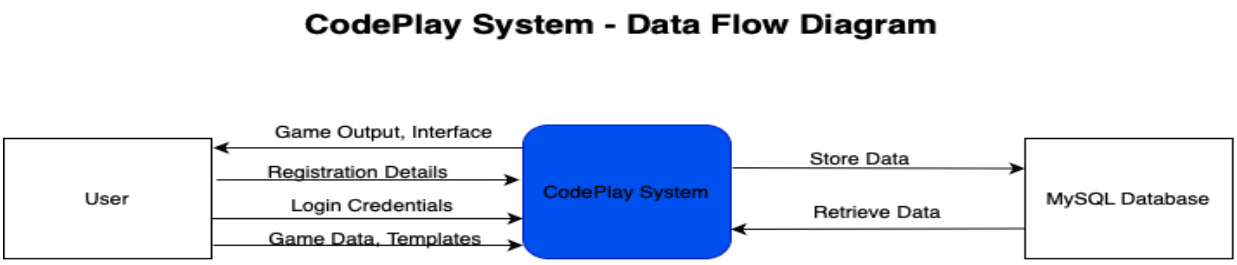
6. Design (DFD Diagram)

Data Flow Diagrams (DFDs) provide a visual representation of data movement within the CodePlay system, illustrating how information flows between external entities, processes, and data stores. This section presents two DFDs: a Level-0 Context Diagram, which shows the system's interaction with external entities, and a Level-1 DFD, which decomposes the system into detailed processes. Since no graphical DFDs were initially provided, the diagrams are defined using PlantUML code, which can be rendered as images for inclusion in the final report. The diagrams are designed to reflect the system's functionality, including user authentication, game template selection, game editing, management, and execution.

Level-0 DFD (Context Diagram)

The Level-0 DFD, or Context Diagram, illustrates the CodePlay system as a single process interacting with two external entities: the User and the MySQL Database. It highlights the high-level data flows, providing an overview of how users interact with the system and how data is stored or retrieved.

- **External Entities**:
  - **User**: Represents individuals who register, log in, create games, and manage their projects.
  - **MySQL Database**: Stores user profiles and game data, serving as the persistent storage mechanism.
- **Data Flows**:
  - **User → CodePlay System**:
    - Registration details (name, email, password).
    - Login credentials (email, password).
    - Game data (title, template type, code).
    - Template selection and management actions (edit, delete).
  - **CodePlay System → User**:
    - Login status (success or failure messages).
    - Game templates with starter code.
    - Editor interface with live preview.
    - List of saved games and game execution output.
  - **CodePlay System ↔ MySQL Database**:
    - Store user profiles and game data.
    - Retrieve user and game data for display or editing.



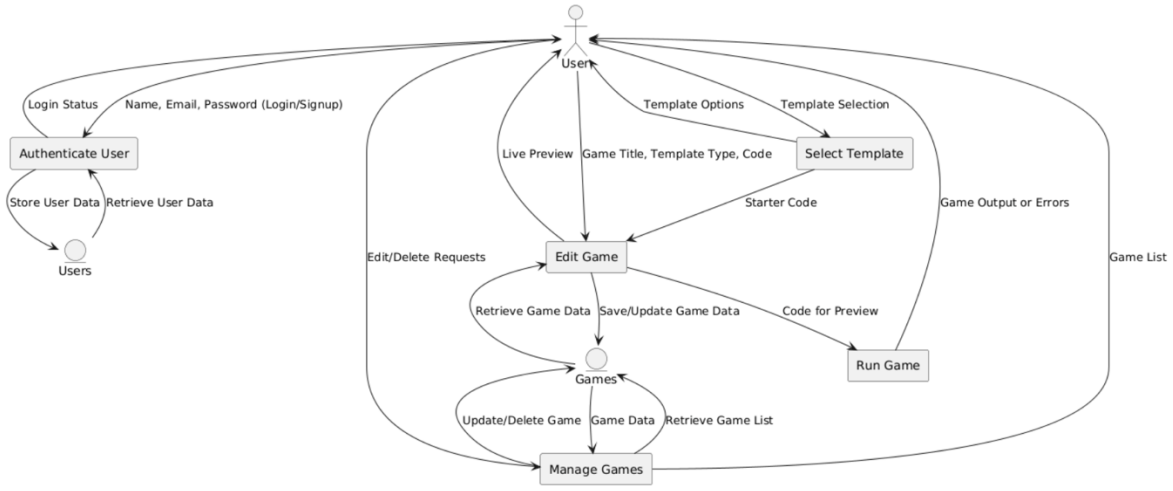
Description: This diagram depicts the CodePlay System as a central rectangle, with arrows indicating data flows to and from the User (actor) and MySQL Database (entity). The labels on the arrows specify the type of data exchanged, such as "Registration Details" or "Game Output." The diagram is simple, focusing on the system's external interactions.

Level-1 DFD

The Level-1 DFD decomposes the CodePlay System into five key processes, detailing how data flows between these processes, the User, and data stores. It provides a more granular view of the system's internal operations, including authentication, template selection, game editing, management, and execution.

- **Processes**:
    1. **Authenticate User**: Validates login or signup inputs, stores new user data, or retrieves existing data for authentication, setting session variables.
    2. **Select Template**: Displays available templates (Maze Runner, Jump Adventure, Catch the Fruit) and provides starter code for the selected template.
    3. **Edit Game**: Accepts user inputs for game title, template type, and code, previews the game in a canvas, and saves or updates game data.
    4. **Manage Games**: Retrieves and displays the user's saved games, handling edit or delete requests.
    5. **Run Game**: Executes saved game code in a canvas, displaying output or error messages.
- **Data Stores**:
    - **Users**: Stores user ID, name, email, and hashed password.
    - **Games**: Stores game ID, user ID, title, template type, code, and timestamps (created_at, updated_at).

- **Data Flows**:
    - **User → Authenticate User**: Name, email, password for signup; email, password for login.
    - **Authenticate User ↔ Users**: Store new user data or retrieve for validation.
    - **User → Select Template**: Template selection (e.g., maze_runner).
    - **Select Template → Edit Game**: Starter code for the selected template.
    - **User → Edit Game**: Game title, template type, code.
    - **Edit Game ↔ Games**: Save or retrieve game data.
    - **Edit Game → Run Game**: Code for preview or execution.
    - **User → Manage Games**: Edit or delete requests.
    - **Manage Games ↔ Games**: Retrieve game list or update/delete records.



Description: The Level-1 DFD shows five processes as rectangles, connected by labeled arrows indicating data flows. The User interacts with each process, while the Users and Games data stores are accessed by relevant processes. For example, the "Edit Game" process sends game data to the Games store and receives starter code from the "Select Template" process. The diagram clarifies the system's internal workflow, highlighting how data moves from user input to storage and output.

7. Implementation

The CodePlay platform is implemented as a full-stack web application, combining server-side and client-side technologies to deliver a seamless game development experience. The implementation leverages PHP for backend logic, MySQL for data storage, and a Bootstrap-based front-end for responsive design. Below is a detailed analysis of the implementation, organized by the provided files and key features.

File Analysis

1. **config.php**:
    - **Purpose**: Configures the MySQL database connection using PDO.
    - **Details**: Connects to the codeplay database on localhost with root credentials (password empty for local setups). Sets PDO options for error handling, fetch mode, and prepared statements to enhance security.
    - **Significance**: Centralizes database configuration, ensuring consistent and secure access across all PHP files.
2. **CG.php**:
    - **Purpose**: Displays a game template selection page for authenticated users.
    - **Details**: Checks for an active session; redirects to login.html if not logged in. Presents three templates (Maze Runner, Jump Adventure, Catch the Fruit) as Bootstrap cards with images and "Start Building" buttons linking to editor.php with a template_type parameter.
    - **Significance**: Serves as the entry point for game creation, guiding users to the editor with a chosen template.
3. **editor.php**:
    - **Purpose**: Provides a code editor interface for writing, previewing, and saving games.
    - **Details**: Includes fields for game title, template type (dropdown), and code (textarea). Supports editing existing games by fetching data from the database. A canvas previews the game when the "Run Code" button is clicked, and "Save Game" stores the data. Uses a background video (nature.mp4) and Bootstrap for styling.
    - **Significance**: Central to the platform, enabling users to create and test games in a single interface.

4. **index.php**:
    - **Purpose**: Acts as the main routing file, handling all pages and form submissions.
    - **Details**: Defines routes for home, login, signup, dashboard, editor, my_games, run_game, and logout. Implements form handling for login, signup, editor submissions, and game deletion. Includes reusable outputHeader and outputFooter functions for consistent layout. Embeds starter code for templates, executed in the editor or run page.
    - **Significance**: The backbone of the application, orchestrating navigation and core functionality.

5. **login.php**:
    - **Purpose**: Processes login form submissions.
    - **Details**: Validates email and password, retrieves user data from the database, and verifies the password using password_verify(). Sets session variables (user_id, user_name) on success and redirects to CG.php.
    - **Significance**: Ensures secure access to the platform's features.
6. **login.html**:
    - **Purpose**: Renders the login form.
    - **Details**: Includes email and password fields with validation (minimum 8 characters). Uses Bootstrap for styling, a background video (cyberpunk-2077-nighttime-metropolis.mp4), and a navbar with the CodePlay logo. Links to signup.html for new users.
    - **Significance**: Provides a professional and engaging entry point for users.
7. **logout.php**:
    - **Purpose**: Terminates user sessions.
    - **Details**: Clears session variables, destroys the session, and redirects to login.html.
    - **Significance**: Ensures secure session management.

8. **run_game.php**:
    - **Purpose**: Handles game code execution and saving from the editor.
    - **Details**: Saves game data (title, template type, code) to the database if the action is "save." Displays the executed code output or a success message. Uses Bootstrap for alerts and styling.
    - **Significance**: Bridges the editor and gameplay, enabling users to test and store their work.
9. **signup.php**:
    - **Purpose**: Processes signup form submissions.
    - **Details**: Validates name, email, and password, checks for duplicate emails, hashes the password, and inserts the user into the database. Redirects to login.html on success.
    - **Significance**: Enables new users to join the platform securely.
10. **signup.html**:
    - **Purpose**: Renders the signup form.
    - **Details**: Includes fields for name, email, and password with validation (minimum 8 characters, uppercase, lowercase, numbers). Styled similarly to login.html with a background video and navbar.
    - **Significance**: Facilitates user onboarding with clear requirements.
11. **my_games.php**:
    - **Purpose**: Displays and manages user's saved games.
    - **Details**: Lists games as cards with title, template type, and timestamps. Offers edit and delete options, with confirmation for deletions. Displays a message if no games exist, linking to CG.php.
    - **Significance**: Provides a centralized hub for game management.

12. **Untitled-1.txt**:
    - **Purpose**: Contains the project's URL.
    - **Details**: Lists http://localhost/library/WT_Project/login.html, indicating the local development entry point.
    - **Significance**: Confirms the project's local hosting setup.
13. **cg.html** and **editor.html**:
    - **Purpose**: Likely placeholders or incomplete files.
    - **Details**: Both are empty, suggesting they may be redundant or unfinished.
    - **Significance**: Highlights potential gaps in the project.

Key Implementation Features

1. **Authentication**:
    - Uses PHP sessions to track logged-in users.
    - Passwords are hashed with password_hash() and verified with password_verify().
    - Protected pages (CG.php, editor.php, my_games.php, run_game.php) check for $_SESSION['user_id'].
2. **Game Templates**:
    - Three templates are defined in index.php with JavaScript code:
        - **Maze Runner**: Navigates a player through a maze to reach a goal, with collision detection.
        - **Jump Adventure**: A platformer where players jump to collect items, with gravity and scoring.
        - **Catch the Fruit**: Players move a basket to catch falling fruits, with lives and scoring.
    - Templates use a canvas element and support keyboard controls (arrow keys, WASD, space).
3. **Code Editor**:
    - The editor (editor.php, index.php?page=editor) provides a textarea for code input, with a canvas for preview.
    - Code execution uses eval() or new Function() in index.php, capturing keyboard inputs via window.gameInput.
    - Supports saving new games or updating existing ones, with validation for required fields.
4. **Game Management**:
    - my_games.php fetches games from the database, displaying them as cards or a table.
    - Edit links to editor.php with the game ID, while delete actions use POST requests with confirmation.
5. **Database Integration**:
    - Uses PDO for secure queries, with prepared statements to prevent SQL injection.
    - Stores user and game data in the users and games tables, respectively.

Technologies

- **Back-End**:
  - PHP 7.4+ for server-side logic, handling routing, form processing, and database interactions.
  - MySQL 5.7+ for persistent storage, accessed via PDO.
- **Front-End**:
  - HTML5 for structure, CSS3 for styling, JavaScript for interactivity.
  - Bootstrap 5.3 for responsive design, including navbar, cards, forms, and alerts.
- **Assets**:
  - Images: logo.png, Maze_Runner.jpg, Jump_Adventure.jpg, Catch_the_Fruit.jpg.
  - Videos: nature.mp4 (editor background), cyberpunk-2077-nighttime-metropolis.mp4 (login/signup background).

Implementation Challenges

- **JavaScript Execution**: Using eval() or new Function() for user code poses security risks, as malicious code could harm the client or server.
- **Missing Files**: Empty cg.html and editor.html suggest incomplete development or redundant files.
- **Error Handling**: Limited feedback for code execution errors in run_game.php, which may confuse users.
- **Performance**: Background videos may slow page loads on low-bandwidth connections.

Testing and Validation

- **Unit Testing**: Test individual functions (e.g., handleLogin, handleSignup in index.php) for edge cases like empty inputs or invalid emails.
- **Integration Testing**: Verify interactions between components, such as form submissions and database updates.
- **User Testing**: Conduct usability tests with target users (e.g., students, hobbyists) to identify pain points in the editor or navigation.
- **Security Testing**: Perform penetration testing to check for vulnerabilities like SQL injection, XSS, or session hijacking.

The implementation demonstrates a solid foundation for a game development platform, with opportunities for enhancement in security, usability, and scalability.

8. Database Design

The CodePlay platform relies on a MySQL database named codeplay to store user profiles and game data. The database is designed to be simple yet effective, supporting the platform's core functionalities. Below is a detailed description of the database schema, relationships, and implementation considerations.

Database Schema

The database consists of two tables: users and games.

Table: users

Column	Type	Description	Constraints
id	INT	Primary key, auto-incremented	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	User's full name	NOT NULL
email	VARCHAR(255)	User's email address	NOT NULL, UNIQUE
password	VARCHAR(255)	Hashed password	NOT NULL

- **Purpose**: Stores user account information for authentication and personalization.
- **Usage**: Queried during login (login.php), signup (signup.php), and session validation.

Table: games

Column	Type	Description	Constraints
id	INT	Primary key, auto-incremented	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	Foreign key referencing users(id)	NOT NULL, FOREIGN KEY
game_title	VARCHAR(255)	Title of the game	NOT NULL
template_type	VARCHAR(255)	Template type (e.g., maze_runner)	NOT NULL
code	TEXT	JavaScript/HTML code of the game	NOT NULL
created_at	DATETIME	Timestamp of game creation	NOT NULL
updated_at	DATETIME	Timestamp of last update	NOT NULL

- **Purpose**: Stores user-created games, linking them to the creator's account.
- **Usage**: Queried in editor.php, my_games.php, run_game.php for saving, listing, and executing games.

Relationships

- **One-to-Many**: One user (users.id) can create multiple games (games.user_id).
- **Foreign Key Constraint**: games.user_id references users.id with ON DELETE CASCADE, ensuring games are deleted if the user account is removed.

Sample SQL Schema

```
CREATE DATABASE codeplay;
USE codeplay;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);



CREATE TABLE games (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    game_title VARCHAR(255) NOT NULL,
    template_type VARCHAR(50) NOT NULL,
    code TEXT NOT NULL,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Implementation Details

- **Connection**: config.php uses PDO to connect to the database, with error handling for connection failures.
- **Security**: All queries use prepared statements to prevent SQL injection. For example, login.php uses SELECT ... WHERE email = ? with bound parameters.
- **Timestamps**: created_at and updated_at are set to NOW() during game creation or updates, ensuring accurate tracking of modifications.
- **Data Types**:
    - VARCHAR(255) for name, email, and game_title accommodates typical input lengths.
    - TEXT for code supports large JavaScript/HTML snippets.
    - VARCHAR(50) for template_type is sufficient for predefined values (maze_runner, jump_adventure, catch_the_fruit, custom).

Query Examples

1. **User Login** (login.php):
2. SELECT id, name, password FROM users WHERE email = ?
    - Binds the user's email to verify credentials.
3. **Save Game** (run_game.php):
4. INSERT INTO games (user_id, game_title, template_type, code, created_at, updated_at)
5. VALUES (?, ?, ?, ?, NOW(), NOW())
    - Inserts a new game with user-provided data.
6. **List Games** (my_games.php):
7. SELECT id, game_title, template_type, created_at, updated_at
8. FROM games
9. WHERE user_id = ?
10. ORDER BY updated_at DESC
    - Retrieves user's games, sorted by last update.
11. **Update Game** (editor.php):
12. UPDATE games
13. SET game_title = ?, template_type = ?, code = ?, updated_at = NOW()
14. WHERE id = ? AND user_id = ?
    - Updates an existing game, ensuring it belongs to the user.

Database Optimization

- **Indexes**:
    - Add an index on games.user_id to speed up queries filtering by user.
    - Index games.updated_at for efficient sorting in my_games.php.
- CREATE INDEX idx_user_id ON games(user_id);
- CREATE INDEX idx_updated_at ON games(updated_at);
- **Partitioning**: For large datasets, partition the games table by user_id to improve query performance.
- **Normalization**: The current design is normalized (no redundant data), with user_id linking games to users.

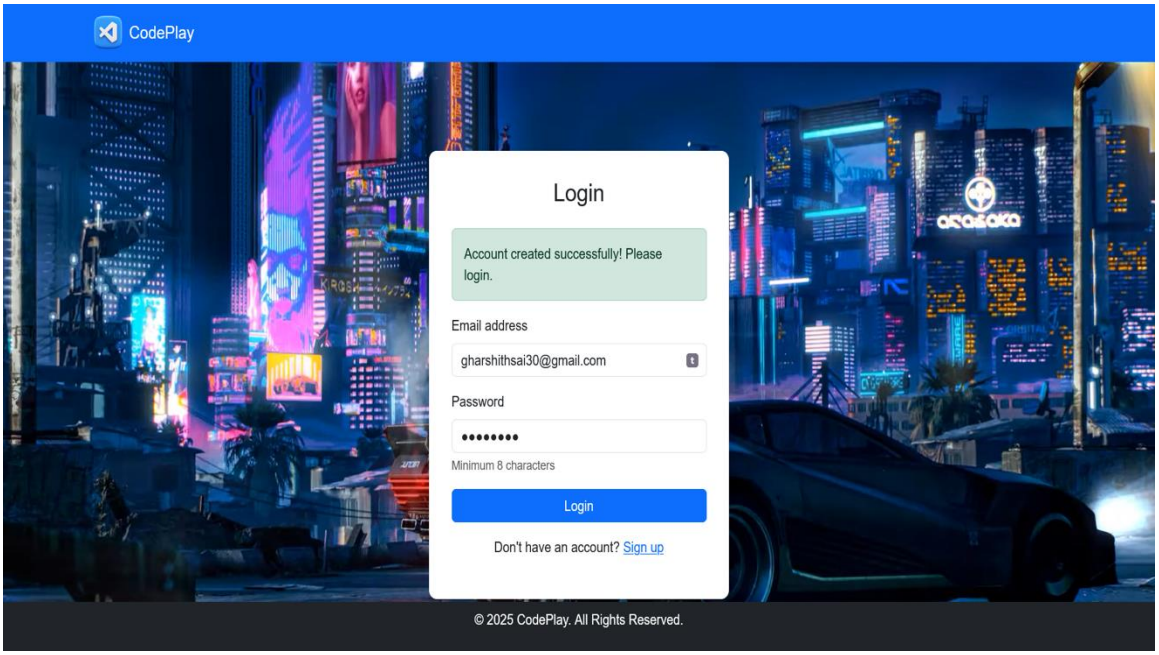
Potential Enhancements

- **Additional Tables**:
    - **Templates**: Store template metadata (e.g., name, description, starter code) to support dynamic template addition.
    - **Game History**: Track game revisions to allow undoing changes.
- **Constraints**:
    - Add a unique constraint on games(user_id, game_title) to prevent duplicate titles per user.
    - Validate template_type with a check constraint for valid values (maze_runner, jump_adventure, catch_the_fruit, custom).
- **Backup and Recovery**:
    - Implement regular database backups to prevent data loss.
    - Use transactions for critical operations (e.g., game saving) to ensure data integrity.

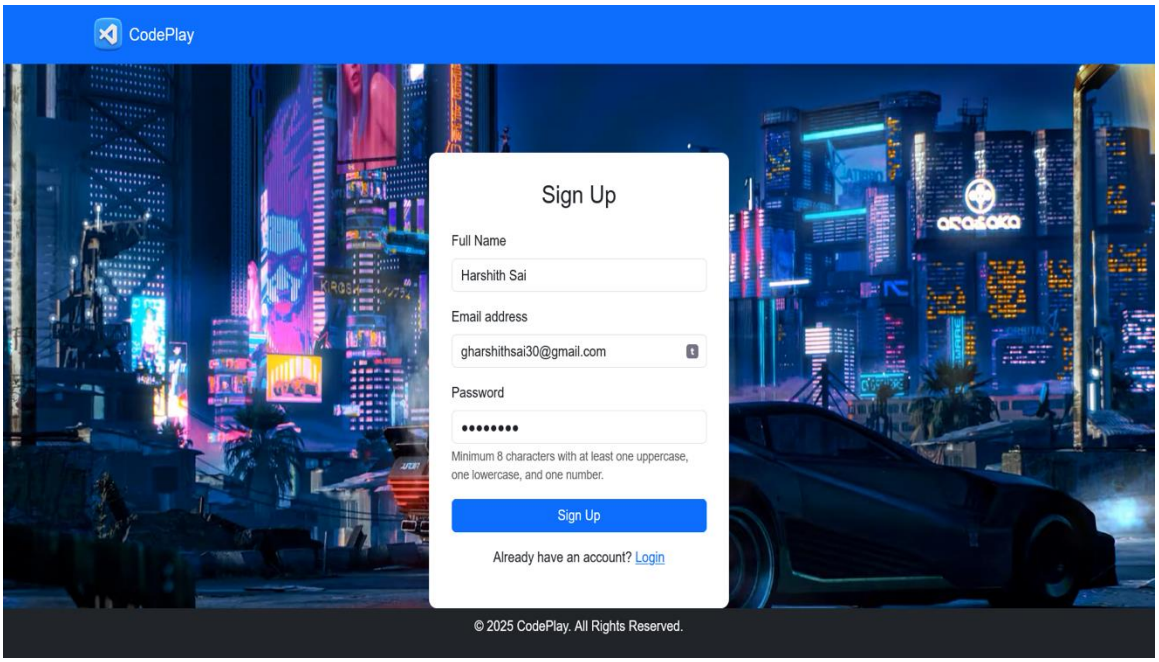
The database design is robust and efficient for the platform's current needs, with room for scalability and additional features as the user base grows.

9. Output Screenshots

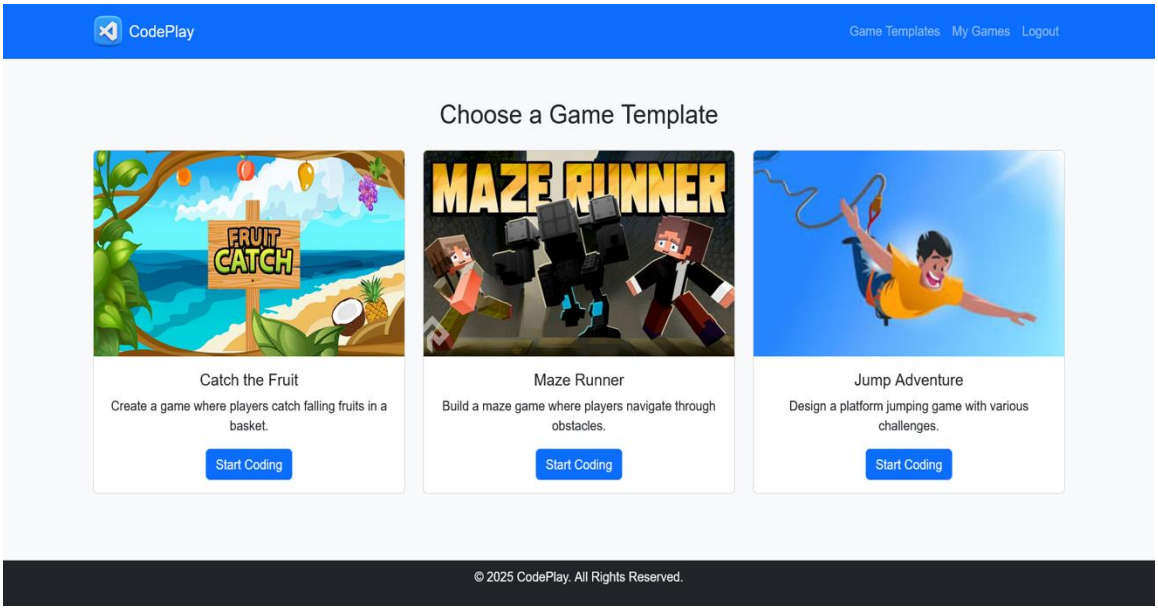
1. **Login Page (**login.html**)**:
    - **Description**: A centered form with email and password fields, styled with Bootstrap. A fixed navbar displays the CodePlay logo and name. A full-screen background video (cyberpunk-2077-nighttime-metropolis.mp4) adds a futuristic aesthetic. A footer shows the copyright notice.
    - **Features**: Email input with type validation, password input with 8-character minimum, and a “Sign up” link.



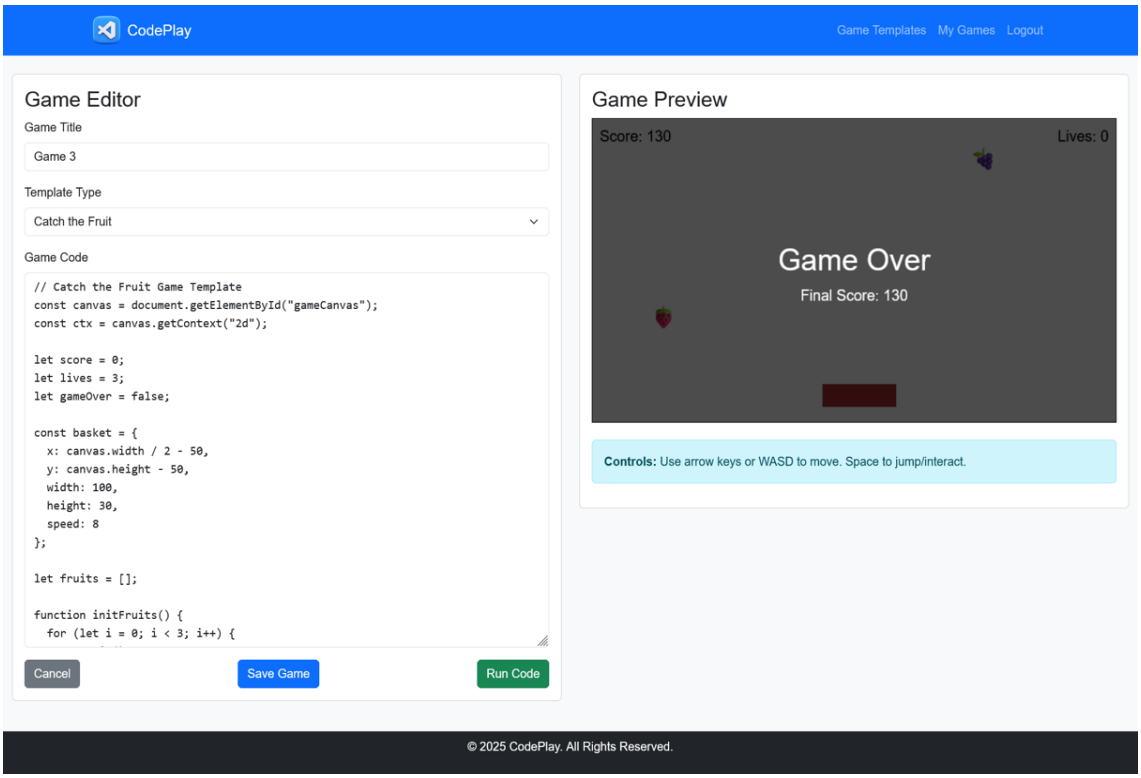
2. **Signup Page (**signup.html**)**:
    - **Description**: Similar to the login page, with fields for name, email, and password. Includes validation text for password requirements (8 characters, uppercase, lowercase, numbers). The same background video and navbar are used.
    - **Features**: Clear form layout, responsive design, and a “Login” link for existing users.



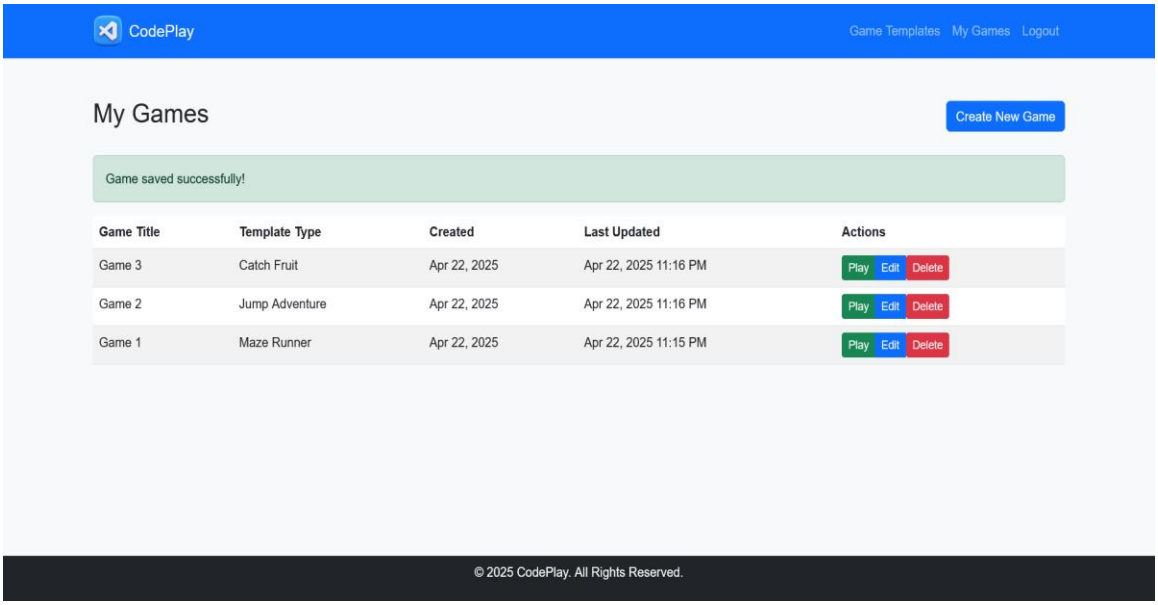
3. **Game Templates Page (**CG.php**)**:
    - **Description**: Displays three Bootstrap cards for Maze Runner, Jump Adventure, and Catch the Fruit, each with an image (Maze_Runner.jpg, etc.), title, and “Start Building” button. The navbar includes links to templates, my games, and logout.
    - **Features**: Hover effects on cards (scale transform), responsive grid layout.



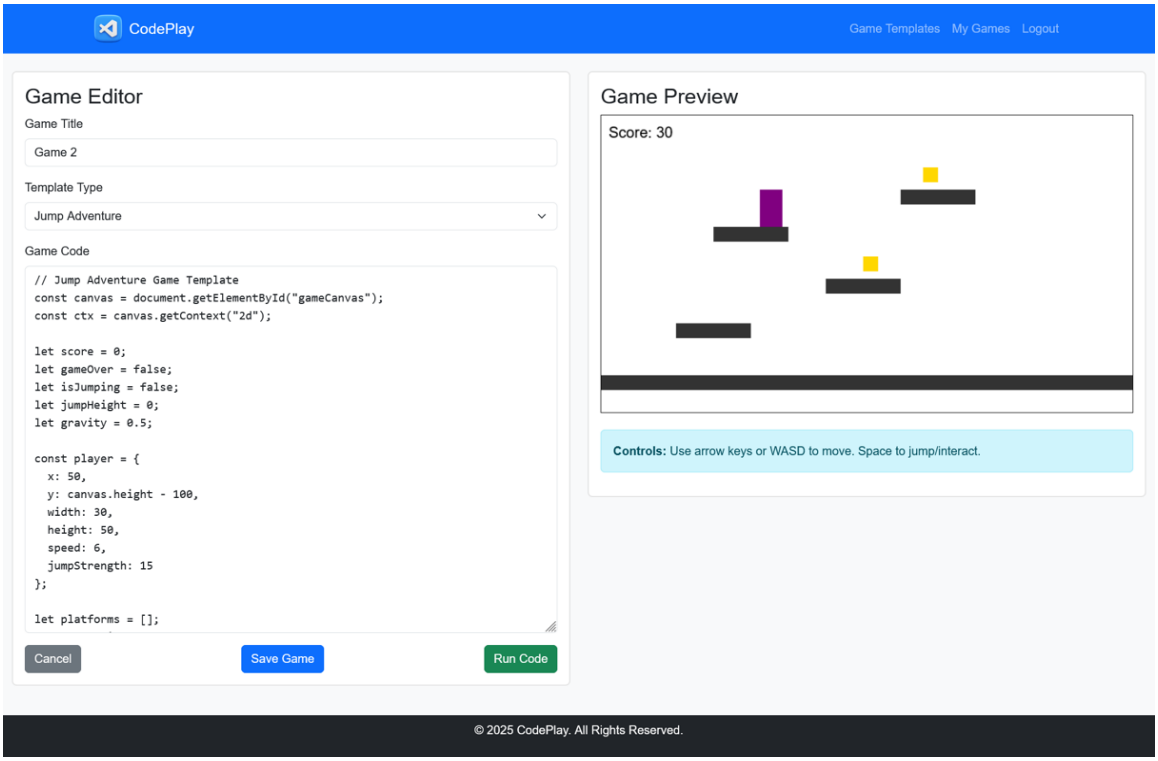
4. **Game Editor (**editor.php**)**:
    - **Description**: A split layout with a code editor (left) and canvas preview (right). The editor includes fields for game title, template type (dropdown), and code (textarea). Buttons allow running or saving the code. A background video (nature.mp4) enhances the aesthetic.
    - **Features**: Real-time preview, responsive flexbox layout, form validation.



5. **My Games (**my_games.php**)**:
    - **Description**: Displays user's games as cards or a table, showing title, template type, creation/update dates, and edit/delete buttons. An alert appears if no games exist, linking to CG.php.
    - **Features**: Confirmation prompt for deletions, success messages for actions.



6. **Game Preview (**run_game.php**)**:
    - **Description**: Shows the executed game code, typically in a canvas (from index.php?page=run_game). Includes a success message for saved games and an error display if execution fails.
    - **Features**: Clean layout, Bootstrap alerts, back-to-games link.



10. Conclusion

CodePlay is a well-executed web application that successfully delivers an accessible and educational game development platform. Its integration of user authentication, game templates, a code editor, and database storage creates a cohesive experience for beginners and hobbyists. The use of modern technologies (PHP, MySQL, Bootstrap) ensures reliability and a professional interface, while the template-based approach fosters learning through practical application.

However, the platform has areas for improvement, particularly in security (JavaScript execution, CSRF), usability (editor enhancements, tutorials), and performance (asset optimization). Addressing these issues will enhance its robustness and user satisfaction. Future enhancements could include:

- Additional templates (e.g., puzzle games, shooters).
- Multiplayer features using WebSockets.
- Integration with a game engine like Phaser for advanced functionality.
- Mobile app versions for iOS and Android.

By continuing to refine and expand its capabilities, CodePlay has the potential to become a leading educational tool in game development, empowering users to create and share their creations with a global community.